PROJECT REPORT

Computer Vision-Guided Plant Spraying Robotic System

Author

SURAJ KUMAR

Precision Agriculture • Computer Vision • Embedded Robotics

Teleoperated mobile spraying platform integrating OpenCV-based target detection, a 2-DOF pan/tilt nozzle gimbal and infrared remote chassis control.

Table of Contents

1. Executive Summary & Project Need

The objective of this project was to plan, design, fabricate, and program a mobile agricultural robot capable of targeted fluid application on foliage while being maneuvered via a custom remote controller.

Automating precision spraying in agricultural robotics reduces chemical waste and limits environmental runoff. This design addresses that need through a dual-subsystem framework: a real-time computer vision system that detects plant targets and aims a 2-DOF pan-tilt nozzle, paired with a teleoperated mobile base controlled by a multi-speed analog joystick via infrared (IR) communication.

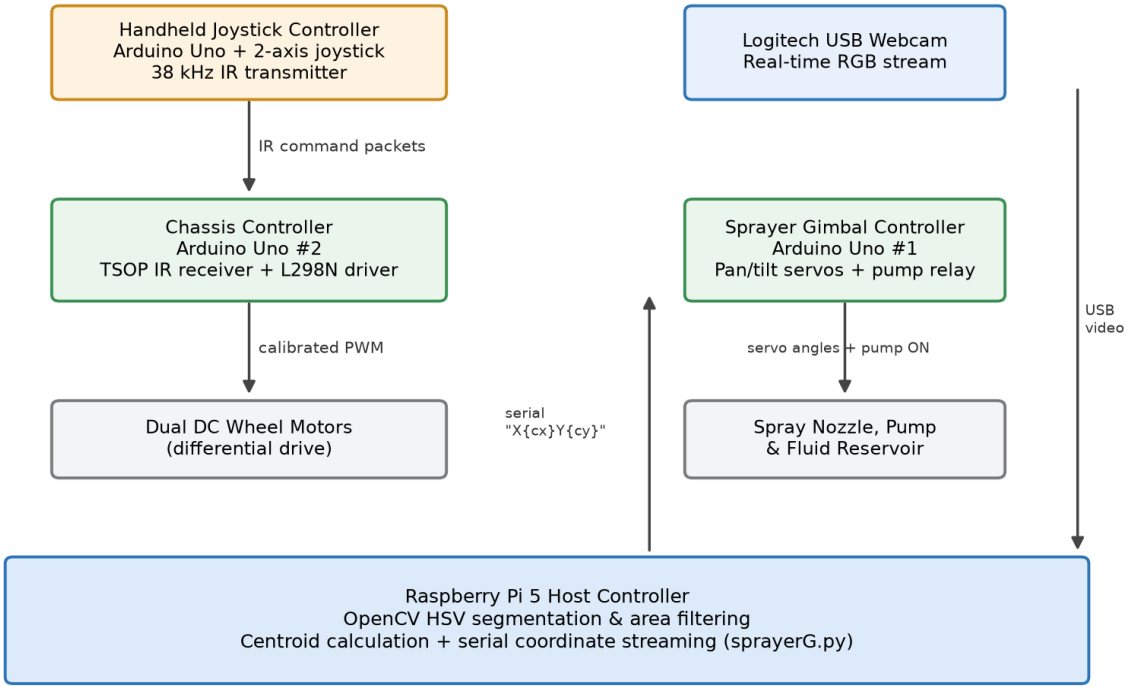

Figure 1: System architecture and signal flow between the host controller, microcontrollers and actuators.

2. System Architecture & Development Strategy

The hardware and computational load is distributed across a single board computer and two dedicated microcontrollers:

- **Vision & Targeting Engine (Raspberry Pi 5):** captures real-time camera frames, executes colour space thresholding to locate green foliage, filters noise, and computes centroid target coordinates.

- **Nozzle Positioning Subsystem (Arduino Uno #1):** receives target (X, Y) coordinate frames over serial from the Raspberry Pi, maps them to rotational angles, drives two servo motors, and triggers the spray pump.
- **Chassis & Drive Subsystem (Arduino Uno #2):** decodes incoming IR signals from the remote transmitter and drives the dual DC wheel motors using calibrated PWM values.
- **Remote Joystick Unit:** uses an Arduino Uno powered by a 9 V battery to read continuous analog voltages from a 2-axis joystick, translates them into 48 directional and speed states, and broadcasts matching IR commands.

3. Hardware Implementation & Pin Specifications

The system utilises an isolated power architecture and dedicated signal pins across the sub-assemblies:

Subsystem	Hardware Component	Interface Pin / Channel	Functional Description
Vision	Logitech USB Webcam	Raspberry Pi USB 3.0	Real-time optical video acquisition
Gimbal	Pan servo motor	Arduino #1 — Digital Pin 9	Horizontal spray angle positioning
Gimbal	Tilt servo motor	Arduino #1 — Digital Pin 10	Vertical spray angle positioning
Actuation	DC water pump relay	Arduino #1 — Digital Pin 7	Fluid pump power activation
Serial link	Pi-to-Arduino bridge	USB serial (/dev/ttyACM0)	Coordinate transfer at 9600 / 115200 baud
Chassis	TSOP IR receiver	Arduino #2 — Digital Pin 2	Decodes remote transmission codes
Chassis	L298N motor driver	Arduino #2 — Pins 3, 4, 5, 6, 7, 8	Differential speed and direction control
Remote	2-axis joystick module	Remote Arduino — Pins A0, A1	Reads analog X and Y control vectors
Remote	IR transmitter LED	Remote Arduino — Digital Pin 3	Emits 38 kHz modulated command codes
User interface	16x2 LCD display	Chassis I2C / digital pins	Displays project name and author initials

Table 1: Component-to-pin allocation across the four sub-assemblies.

4. Software Architecture & Implementation Flow

The processing chain runs from frame capture on the Raspberry Pi through to servo actuation and pump triggering on the gimbal controller:

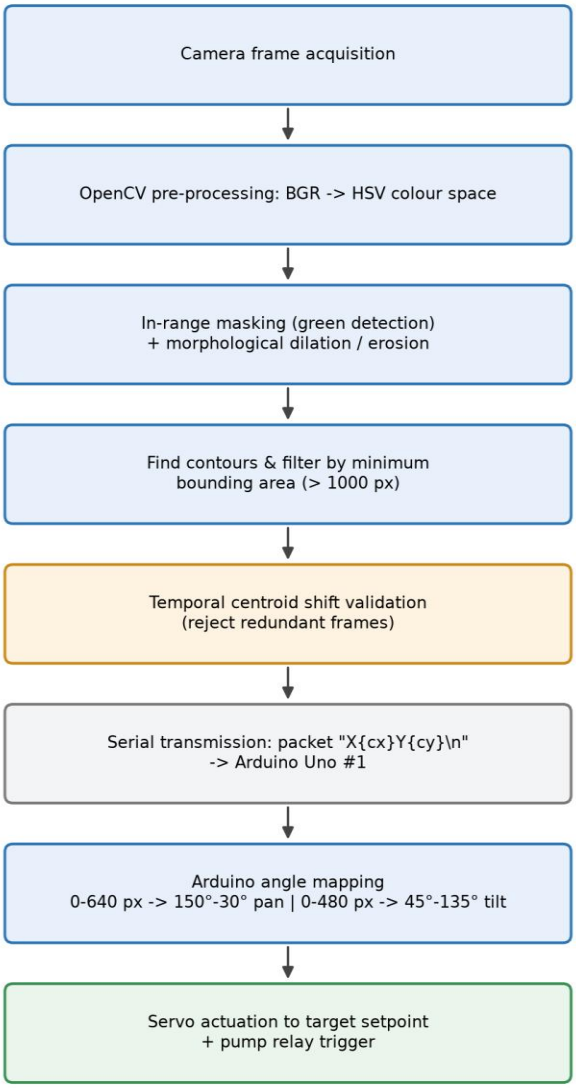

Figure 2: Vision-to-actuation processing flowchart.

4.1 Software Modules

- **sprayerG.py (Raspberry Pi vision script):** connects to the camera via OpenCV, processes frames through HSV threshold boundaries, applies morphological open/close filtering, and extracts the target centroid. Implements positional caching (last_x, last_y) and area thresholds to discard noise before dispatching coordinates across the serial link.
- **Sprayer.ino (gimbal actuation firmware):** parses incoming serial strings (X...Y...), extracts numerical coordinate substrings, scales values to bounded angular limits (PAN_MIN/PAN_MAX and TILT_MIN/TILT_MAX), and updates servo positioning while activating the pump circuit.

- **MainRobotCode.ino (chassis drive firmware):** decodes IR pulses using the IRremote library and controls the motor driver. Uses hardcoded PWM offset compensation to balance motor output differences, ensuring straight line tracking without encoder feedback.
- **Remote.ino (joystick transmitter firmware):** polls analog pins A0 and A1, checks against a central deadzone, maps joystick deflection into one of 48 operational states (including forward slow, medium, fast, reverse and turning), and sends the corresponding hex command via the IR LED.

5. Technical Challenges & Engineering Procedural Adjustments

Challenge Encountered	Procedural Adjustment / Engineering Fix
Servo voltage drop & serial crashes (termios.error: 5 Input/Output)	Isolated servo power to an external battery source rather than drawing from the Arduino 5 V rail.
Linux Python package dependencies	Resolved environment conflicts on Pi OS by installing dependencies using python3 package management flags.
Chassis weight overload & drift imbalance	Bypassed wheel encoders and applied fixed, calibrated PWM speed trims per drive state.
Fluid delivery & wiring faults	Resoldered power/ground leads and deployed an onboard tube feed to a portable reservoir.
Bounding box multi-detection noise	Added minimum contour area filters and temporary coordinate check variables on the Pi.

Table 2: Summary of engineering issues and the corrective actions applied.

5.1 Detailed Discussion

- **Voltage supply to servos:** drawing current for the pan/tilt servo motors directly from the Arduino Uno's onboard voltage regulator caused brownouts, resetting the microcontroller and crashing the Python serial pipe with termios.error: (5, 'Input/output error'). This was resolved by separating the logic and actuator power rails, supplying the servos from an external battery pack with a common ground.
- **Raspberry Pi environment configuration:** initial package installation failures were resolved by configuring correct user permissions and installing the required system libraries through python3 package management flags.
- **Chassis weight & encoder failure:** adding the sprayer pump, fluid lines, dual microcontrollers and battery packs increased the robot's weight, causing wheel encoders to slip and misread. The team shifted to open-loop manual teleoperation, manually tuning PWM speeds across slow, medium and fast modes so the robot tracked straight.
- **Fluid reservoir integration:** solved fluid tube routing and connection issues by resoldering faulty pump power wires and routing the intake tube into an onboard mobile water container.

- **Bounding box chatter & false positives:** ambient lighting and minor colour variations generated multiple flickering bounding boxes. The Python vision script was updated to enforce a minimum contour area threshold and use delta coordinate checks, ignoring sub-threshold noise and preventing redundant serial traffic.

6. System Verification & Results

The system successfully demonstrated synchronised multi-board operation during live evaluations:

- **Target acquisition:** the OpenCV vision pipeline successfully isolated green objects, calculated coordinate offsets and sent target updates to the gimbal controller.
- **Gimbal tracking & spray delivery:** the pan/tilt servos tracked the green target across the camera's field of view and triggered the spray pump upon alignment.
- **Remote mobility:** the custom IR joystick controller transmitted 48 distinct movement states, giving the operator proportional control over chassis speed, steering and braking.

7. Future Work & Recommendations

- **Machine learning plant classification:** replace basic HSV colour segmentation with a lightweight object detection model (e.g. YOLOv8-nano or MobileNet SSD) to recognise specific crops or weeds rather than generic green shapes.
- **Microcontroller consolidation:** port the servo and motor PWM generation directly to the Raspberry Pi 5 GPIO pins, removing the need for auxiliary microcontrollers and serial bridging.
- **Autonomous navigation:** upgrade the drive base with higher-torque geared motors and optical encoders, integrating closed-loop PID control to enable autonomous waypoint following.
- **Custom chassis & fluid containment:** replace temporary adhesive mountings with a 3D-printed, weather-resistant enclosure that includes an integrated, baffled fluid reservoir.